

CS2422 Assembly Language & System Programming

October 9, 2003

Today's Topics

- Conditional Processing (The rest of Chapter 6)
 - Conditional Loop
 - .IF, .WHILE, .REPEAT Directives

Study Guide

- Sections 6.4 and 6.7

A Quick Review – CMP and Jcond Instruction

- The IF statement in C and PASCAL is converted into CMP and Jcond instructions in x86 Assembly:

```
If (X > op1)
Then
    <...>
End If
```



```
CMP X, op1
JNG EndIf
    <...>
EndIf:
```

Rest of Chapter 6

- Conditional Loop Instructions.
- Finite State Machine (*not covered today*)
- .IF, .WHILE, and .REPEAT Directives
(note: not Instructions)

Conditional Loop Instructions

- LOOPZ and LOOPE
- LOOPNZ and LOOPNE

LOOPZ and LOOPE

- Syntax:
 LOOPE *destination*
 LOOPZ *destination*
- Logic:
 - $ECX \leftarrow ECX - 1$
 - if $ECX > 0$ and $ZF=1$, jump to *destination*
- Useful when scanning an array for the first element that does not match a given value.

LOOPNZ and LOOPNE

- Syntax:
 LOOPNZ *destination*
 LOOPNE *destination*
- Logic:
 - $ECX \leftarrow ECX - 1$;
 - if $ECX > 0$ and $ZF=0$, jump to *destination*
- Useful when scanning an array for the first element that matches a given value.

LOOPNZ Example

The following code finds the first positive value in an array:

```
.data
array SWORD -3,-6,-1,-10,10,30,40,4
.code
    mov esi,OFFSET array
    mov ecx,LENGTHOF array
    sub esi,TYPE array
next:
    add esi, TYPE array
    test WORD PTR [esi],8000h ; test sign bit
    loopnz next             ; continue loop
    jnz quit                ; none found
    ...                     ; ESI points to value
quit:
```

Using the .IF Directive

- Runtime Expressions
- Relational and Logical Operators
- MASM-Generated Code
- .REPEAT Directive
- .WHILE Directive

Runtime Expressions

- .IF, .ELSE, .ELSEIF, and .ENDIF can be used to evaluate runtime expressions and create block-structured IF statements.
- Examples:

```
.IF eax>ebx
    mov edx,1
.ELSE
    mov edx,2
.ENDIF
```

```
.IF eax>ebx && eax>ecx
    mov edx,1
.ELSE
    mov edx,2
.ENDIF
```

- MASM generates "hidden" code for you, consisting of code labels, CMP and conditional jump instructions.

Relational and Logical Operators

Operator	Description
<i>expr1</i> == <i>expr2</i>	Returns true when <i>expression1</i> is equal to <i>expr2</i> .
<i>expr1</i> != <i>expr2</i>	Returns true when <i>expr1</i> is not equal to <i>expr2</i> .
<i>expr1</i> > <i>expr2</i>	Returns true when <i>expr1</i> is greater than <i>expr2</i> .
<i>expr1</i> >= <i>expr2</i>	Returns true when <i>expr1</i> is greater than or equal to <i>expr2</i> .
<i>expr1</i> < <i>expr2</i>	Returns true when <i>expr1</i> is less than <i>expr2</i> .
<i>expr1</i> <= <i>expr2</i>	Returns true when <i>expr1</i> is less than or equal to <i>expr2</i> .
! <i>expr</i>	Returns true when <i>expr</i> is false.
<i>expr1</i> && <i>expr2</i>	Performs logical AND between <i>expr1</i> and <i>expr2</i> .
<i>expr1</i> <i>expr2</i>	Performs logical OR between <i>expr1</i> and <i>expr2</i> .
<i>expr1</i> & <i>expr2</i>	Performs bitwise AND between <i>expr1</i> and <i>expr2</i> .
CARRY?	Returns true if the Carry flag is set.
OVERFLOW?	Returns true if the Overflow flag is set.
PARITY?	Returns true if the Parity flag is set.
SIGN?	Returns true if the Sign flag is set.
ZERO?	Returns true if the Zero flag is set.

MASM-Generated Code

```
.data
val1    DWORD 5
result  DWORD ?
.code
mov eax,6
      →
  .IF eax > val1
    mov result,1
  .ENDIF
```

Generated code:

```
mov eax,6
cmp eax,val1
jbe @C0001
mov result,1
@C0001:
```

MASM automatically generates an unsigned jump (JBE).

MASM-Generated Code

```
.data
val1    SDWORD 5
result  SDWORD ?
.code
mov eax,6
      →
  .IF eax > val1
    mov result,1
  .ENDIF
```

Generated code:

```
mov eax,6
cmp eax,val1
jle @C0001
mov result,1
@C0001:
```

MASM automatically generates a signed jump (JLE).

.REPEAT Directive

Executes the loop body before testing the loop condition associated with the .UNTIL directive.

Example:

```
; Display integers 1 - 10:

mov eax,0
.REPEAT
    inc eax
    call WriteDec
    call Crlf
.UNTIL eax == 10
```

.WHILE Directive

Tests the loop condition before executing the loop body
The .ENDW directive marks the end of the loop.

Example:

```
; Display integers 1 - 10:

mov eax,0
.WHILE eax < 10
    inc eax
    call WriteDec
    call Crlf
.ENDW
```

When to Use or Not to Use Directives?

- Directives make assembly language easier to write and to understand, by hiding tedious work. (*Food for thought: Wouldn't it be even better to use C language?*)
- Don't use directives if you want to have total control.